



Revisiting Network Coding for Warm Blob Storage

Chuang Gan, *Huazhong University of Science and Technology*; Yuchong Hu, *Huazhong University of Science and Technology and Shenzhen Huazhong University of Science and Technology Research Institute*; Leyan Zhao, Xin Zhao, Pengyu Gong, and Dan Feng, *Huazhong University of Science and Technology*

<https://www.usenix.org/conference/fast25/presentation/gan>

This paper is included in the Proceedings of the
23rd USENIX Conference on File and Storage Technologies.

February 25–27, 2025 • Santa Clara, CA, USA

ISBN 978-1-939133-45-8

Open access to the Proceedings
of the 23rd USENIX Conference on
File and Storage Technologies
is sponsored by

 **NetApp®**

Revisiting Network Coding for Warm Blob Storage

Chuang Gan¹, Yuchong Hu^{1,2}, Leyan Zhao¹, Xin Zhao¹, Pengyu Gong¹, and Dan Feng¹

¹*School of Computer Science and Technology, Huazhong University of Science and Technology*

²*Shenzhen Huazhong University of Science and Technology Research Institute*

Abstract

Minimum-storage regenerating (MSR) codes are repair-optimal erasure codes that minimize the bandwidth for repairing a failed node, while minimizing the storage redundancy necessary for fault tolerance. Recent studies in the literature, both from coding theory and systems communities, mainly examine MSR codes in *systematic* form, which keeps the original data blocks as part of the encoded blocks for direct access. However, systematic MSR codes manage encoded blocks at the sub-block granularity and access non-contiguous sub-blocks during repairs to achieve bandwidth optimality. Thus, their actual repair performance is impaired by non-contiguous I/Os, especially when the block size is small. In this paper, we explore how *non-systematic* MSR codes, which generate purely coded blocks based on random linear coding in the classical network coding theory, can improve I/O efficiency in repair for practical warm blob (binary large object) storage systems that are dominated by a large fraction of small blobs. To this end, we design NCBlob, a network-coding-based warm blob storage system that encodes small blobs non-systematic MSR codes to achieve high repair performance, while leveraging the access locality of small blobs to maintain high normal read performance. Experiments on Alibaba Cloud show that NCBlob reduces the single-block repair time by up to 45.0%, and the full-node repair time by up to 38.4%, with as low as 2.1% read throughput loss, compared with state-of-the-art systematic MSR codes.

1 Introduction

Erasure coding has been widely deployed in distributed storage systems that often store occasionally accessed (i.e., warm) [29] or rarely accessed (i.e., cold) [2, 6, 10] data. It encodes the original uncoded *data blocks* into coded *parity blocks* to form a *stripe*, such that the data blocks can be reconstructed from a subset of a sufficient number of data and parity blocks of the same stripe. It significantly saves the storage overhead compared to making exact copies in traditional replication, while maintaining the same degree of fault tolerance [49]. However, erasure coding also incurs high repair bandwidth (i.e., the amount of data transferred during a single-block repair), as repairing any single lost block needs to retrieve multiple available blocks of the same stripe for decoding.

Many research efforts have been made to design new erasure codes with significantly less repair bandwidth. One representative family of erasure codes is *minimum-storage regener-*

ating (MSR) codes [15], which provably minimize the repair bandwidth, while achieving the minimum storage redundancy for tolerating any number of lost blocks. MSR codes divide each block into a number of sub-blocks (the number is called the *sub-packetization level*), such that each node can mix the sub-blocks to send a limited amount of mixed data during a single-block repair. The first construction of MSR codes [15] is based on classical *random linear coding* in the network coding theory [5], in which a stripe is *non-systematic* and contains all parity blocks encoded from data blocks, while the sub-packetization level is polynomial with respect to the number of blocks in a stripe. Prior studies also show that non-systematic MSR codes can be feasibly implemented for cold archival storage [6, 14]. However, since non-systematic MSR codes do not keep data blocks, accessing any data block must first retrieve sufficient parity blocks for decoding. This leads to extra computational and I/O costs in normal reads.

Since MSR codes are proposed, follow-up studies propose new MSR codes in *systematic* form (i.e., data blocks are kept in a stripe), both from coding theory [38, 43] and systems communities [23, 34, 39, 48]. Among them, Clay codes [48] are the state-of-the-art MSR codes with the attractive practical properties: minimum storage redundancy, general coding parameters, and the same total size of disk I/Os as the repair bandwidth. However, unlike the polynomial sub-packetization level in non-systematic MSR codes, Clay codes have a sub-packetization level that increases exponentially with the number of blocks in a stripe (§2.2); this condition is theoretically necessary for the practical properties to be achievable. Having a high number of sub-blocks leads to non-contiguous I/Os during a repair and may even negate the actual repair performance [46], especially for a small block size.

This motivates us to revisit systematic and non-systematic MSR codes to combine the best of both worlds. We make a case for warm blob (binary large object) storage [29, 41], in which read/repair performance and storage efficiency are both critical requirements. To this end, we design NCBlob, a network-coding-based warm blob storage system that adopts a hybrid MSR-coded approach, in which large blobs are encoded with systematic MSR codes, while small blobs are encoded by non-systematic MSR codes. Our insight is that practical warm blob storage workloads are dominated by small blobs (e.g., up to 99% of blobs are smaller than 1 MB [29, 41]), so non-systematic MSR codes mitigate the I/O overhead during a repair. To mitigate the computational and I/O overhead

in normal reads under non-systematic MSR codes, we propose *access locality*, which carefully groups the data units that are likely accessed together to achieve high normal read performance. Our contributions include:

- We observe that blob storage typically has a lot of small blobs and systematic MSR codes perform poorly on small blobs, which motivates us to leverage non-systematic MSR codes to enhance the repair performance of small blobs in warm blob storage (§2).
- We design NCBlob with two non-systematic MSR encoding scheme based on two types of data access localities to reduce read amplification. We also propose a hybrid MSR codes for both small and large blobs to achieve a good overall repair performance. Finally, we prove that NCBlob exists for any coding parameters with tolerating at most three failures (§3).
- We implement NCBlob prototype, and also implement parallelization schemes based on ParaRC [26] to accelerate NCBlob’s repair and read operations (§4). The source code of our prototype is now available at: <https://github.com/YuchongHu/NCBlob>.
- Evaluation results via Alibaba Cloud experiments show that compared to state-of-the-art systematic RC codes, NCBlob significantly reduces the single-block repair time by up to 45.0%, and the full-node repair time by up to 38.4%, with as low as 2.1% read throughput loss (§5).

2 Background and Motivation

2.1 Blob Storage

We consider a blob storage system with the following key characteristics.

- *Blobs are immutable*: Blobs contain binary contents, such as photos, videos, documents, or application installers. Once a blob is created and stored, it is never modified (i.e., immutable). It is read multiple times, and may later be deleted.
- *Multiple access tiers exist*: Practical blob storage systems (e.g., Google Cloud, Amazon AWS, Azure, and Facebook) keep blobs in different performance tiers to address the cost-performance trade-off [12, 29]. For example, Facebook’s blobs become less frequently accessed as they age [29]. Thus, Facebook stores newly written blobs in Haystack with high access performance [11]. As Haystack’s blobs age (e.g., after a month of storage for most blob types) and their access rates drop by orders of magnitude (up to 100x), they are moved to f4 [29], a warm blob storage system with decent access performance. As the blobs age further, they are archived in cold storage for low-cost persistence, while the access performance is of a less concern (e.g., blob retrievals can take days or hours).
- *Warm blobs are common*: Warm blobs in practice account for a majority of blob storage. For example, Facebook’s warm blobs occupy more than 80% of all blob types [29].

- *Small blobs dominate*: Blobs in practice have small sizes. For example, in Facebook’s f4 [29], almost all photo blobs are smaller than 1 MB, while 60% of video blobs are smaller than 1 MB; in Azure, 80% of blobs accessed by Azure functions are smaller than 12 KB [41].

Based on the above analysis, we target warm blob storage that is dominated by small blobs as in Facebook’s f4 [29]. To improve storage efficiency, f4 [29] stores warm blobs in erasure coding (as opposed to replication in Haystack [11]), but the access performance is hindered by the high repair penalty of erasure coding (§2.2). Since warm blobs are still regularly accessed (albeit infrequent), in this paper, we explore new erasure codes that improve access performance, while maintaining high storage efficiency.

2.2 Erasure Coding

Basics. We consider a distributed storage system that organizes data in the fixed-size blocks across nodes and performs erasure coding in units of blocks. Reed-Solomon (RS) codes [40] are the most widely deployed erasure codes [3, 16, 29]. Specifically, an (n, k) RS code is configured by two parameters k and n (where $k < n$). It encodes k fixed-size (uncoded) *data blocks* $D_1, D_2, \dots, D_k$ into $n - k$ (coded) *parity blocks* $P_1, P_2, \dots, P_{n-k}$ of the same size, and the set of n blocks after encoding is called a *stripe*. A storage system contains multiple stripes that are independently encoded, and the n blocks of each stripe are stored in n different nodes. Mathematically, each parity block is a linear combination of the k data blocks based on the Galois Field arithmetic [37], expressed as $P_j = \sum_{i=1}^k a_{j,i} D_i$ ($1 \leq j \leq n - k$), where $a_{j,i}$ denotes some encoding coefficient. RS codes have the following practical properties that make them popular: (i) *storage-optimal* (a.k.a. maximum distance separable (MDS)), i.e., the redundancy is minimum while any k out of the n blocks of each stripe can reconstruct all k data blocks of the stripe (in other words, the distributed storage system can tolerate any $n - k$ node failures); (ii) *general*, i.e., any parameters k and n (where $k < n$) are supported; and (iii) *systematic*, i.e., the k data blocks are included in the stripe after encoding.

RS codes, however, incur amplified *repair bandwidth* (i.e., the amount of transferred data during a single-block repair). Repairing a lost block in an (n, k) RS code retrieves k blocks from k available nodes (called *helpers*) to a *requestor* (a new node to replace a failed node, or a client that requests the lost block). Figure 1(a) shows that the single-block repair of the $(4, 2)$ RS code retrieves two blocks. Note that the above repair also incurs two I/O accesses, one to each helper.

Minimum-Storage Regenerating (MSR) codes. MSR codes [15] are repair-optimal erasure codes that provably minimize the repair bandwidth for repairing a lost block, while preserving the storage optimality of RS codes. Their core idea is *sub-packetization*, which divides each block into α sub-blocks, where α is called the *sub-packetization level*, and performs coding at the sub-block granularity. Repairing any

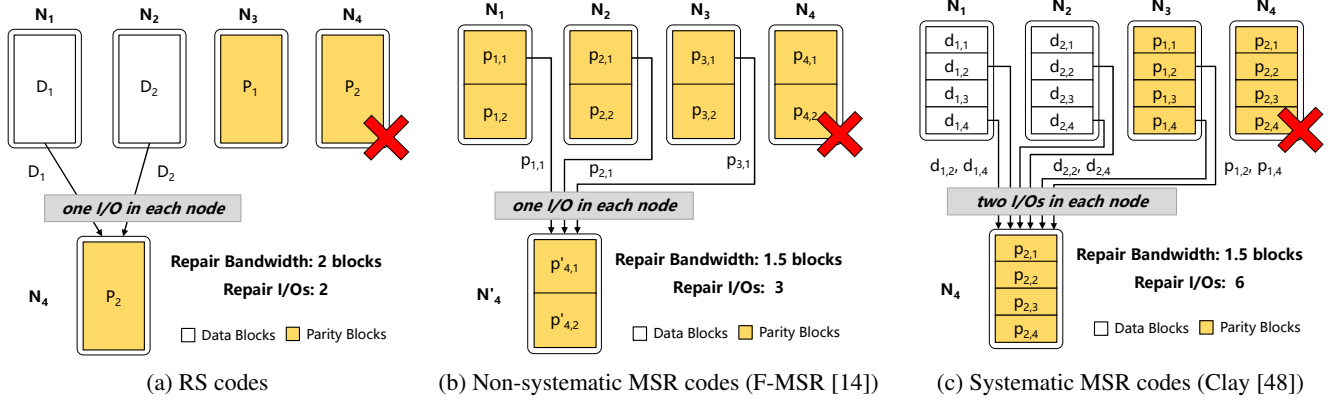


Figure 1: Code constructions and single-block repairs for the (4,2) RS code, (4,2) F-MSR code and (4,2) Clay code. Each sub-figure shows a stripe that is stored in four nodes N_0 , N_1 , N_2 , and N_3 , and N_4 is the requestor.

lost block of a failed node is done by mixing the $(n-1) \times \alpha$ sub-blocks of all $n-1$ available nodes (i.e., helpers) of the same stripe with the minimum repair bandwidth.

MSR codes are first constructed in a *non-systematic* form [15] based on random linear network coding [5], in which each stripe only contains parity blocks that are linear combinations of data blocks (under Galois Field arithmetic) and has a sub-packetization level $\alpha = n-k$. One key feature is that a single-block repair is *functional*, meaning that the repaired block may differ from the originally lost block, as long as fault tolerance is still maintained. One non-systematic MSR code construction is functional MSR (F-MSR) codes [14]. Figure 1(b) depicts the single-block repair of the (4,2) F-MSR code, which contains the parity blocks P_1, P_2, P_3, P_4 and each parity block P_i is divided into $p_{i,1}$ and $p_{i,2}$ ($1 \leq i \leq 4$). The single-block repair (say P_4) forms $p'_{4,1}$ and $p'_{4,2}$ (different from the original $p_{4,1}$ and $p_{4,2}$) by retrieving three sub-blocks $p_{1,1}$, $p_{2,1}$, and $p_{3,1}$ (which incurs three I/O accesses). The repair bandwidth is 1.5 blocks (i.e., 25% less than that of the (4,2) RS code). Note that F-MSR codes are also *access-optimal*, meaning that the total size of the accessed sub-blocks from disk is equal to the repair bandwidth and is also minimized.

Follow-up studies from coding theory [38, 43] and systems communities [23, 34, 39, 48] propose systematic MSR codes, among which Clay codes [48] are state-of-the-art MSR codes that preserve all practical properties of RS codes (i.e., storage-optimal, supporting general (n, k) , and systematic) and are also access-optimal as in F-MSR codes. Clay codes build on pairwise coupling of sub-blocks, and are shown to be feasibly deployable on Ceph [3]. Figure 1(c) depicts that the (4,2) Clay code, which has a sub-packetization level $\alpha = 4$ (see §2.3 for the details on α). The single-block repair retrieves six sub-blocks, so the repair bandwidth is 1.5 blocks as in F-MSR codes. However, the repair also incurs six non-contiguous I/O accesses, which lead to degraded repair performance (§2.3).

Comparisons with locally repairable codes. In addition to MSR codes, *locally repairable codes (LRCs)* [17, 20, 36, 42] are another popular family of erasure codes that leverage local repair to reduce the repair bandwidth. Taken (n, k, r) Azure-

LRC [20] (where $r < k < n$) as an example, it divides k data blocks into $\frac{k}{r}$ local groups with r data blocks each (assuming that k is divisible by r), and encode each local group into a local parity block. It also encodes k data blocks into $n - k - \frac{k}{r}$ global parity blocks, thus tolerates any $n - k - \frac{k}{r} + 1$ node failures [20]. Its repair bandwidth is r blocks for repairing a data block or a local parity block (i.e., local repair), and k blocks for global parity blocks (as in RS codes).

Azure-LRC is *not* storage-optimal. Figure 2 shows the repair bandwidth versus storage redundancy of Azure-LRC and Clay codes. We fix the number of tolerable lost blocks as $f = 2, 3, 4$, and vary k ($f \leq k \leq 20$); for Azure-LRC, we set $r = \frac{k}{2}$. Clay codes outperforms Azure-LRC in terms of the storage-bandwidth trade-off. For example, for $f = 4$, the repair bandwidth of the (15,10,5) Azure-LRC (6 blocks) is 54.2% less than that of the (12,8) Clay code (2.75 blocks), with the same storage redundancy $1.5\times$. Also, the (24,20) Clay code has lower repair bandwidth (5.75 blocks) and lower redundancy ($1.2\times$) compared to the (15,10,5) Azure-LRC. In this work, we focus on MSR codes rather than LRCs since MSR codes provide a better storage-bandwidth trade-off.

2.3 Limitations of Clay Codes

Despite the positives of Clay codes, they have an exponential sub-packetization level $\alpha = (n-k)^{\lceil \frac{n}{n-k} \rceil}$, and the number of sub-blocks accessed by each node in a single-block repair $(n-k)^{\lceil \frac{n}{n-k} \rceil - 1}$ [48]. The exponential sub-packetization level is fundamental in order for Clay codes to maintain both systematic and access-optimal properties [48].

We observe two limitations of high sub-packetization level in the two typical data layouts of blob storage systems:

Limitation #1 (Slow single-block repair under stripe layout). Stripe layout is widely deployed in many object storage systems (e.g., Ceph [3], HDFS [8] and QFS [33]), each blob is basically divided into k blocks and the blocks are encoded into $n-k$ parity blocks with (n, k) Clay code, such that small blobs may be divided into small blocks. In Clay codes, repairing a lost block requires each node to read many sub-blocks of the block from the disk. The ideal situation is all the needed sub-blocks

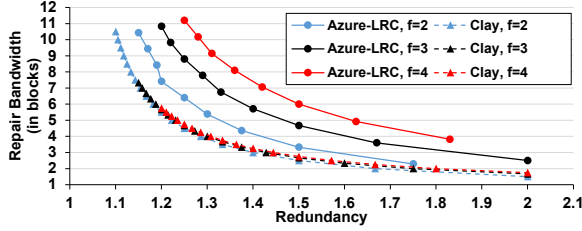


Figure 2: Trade-off analysis between storage redundancy and repair bandwidth for Clay codes and Azure-LRC.

can be read via a contiguous I/O operation (e.g., read $d_{1,1}, d_{1,2}$ in Figure 1(c) only incurs one contiguous I/O). However, we observe that the I/Os to read those sub-blocks are mostly non-contiguous, and in the worst case of the Clay code's repair, only one sub-block can be read in one I/O, such that the number of non-contiguous I/Os equals $(n-k)^{\lceil \frac{n}{n-k} \rceil - 1}$ [48] which follows the exponential increase of α (e.g., the single-block repair in (14, 10) Clay code incurs up to 64 non-contiguous I/Os from each node). Thus, high sub-packetization incurs many non-contiguous I/Os in a block, which seriously impairs the repair performance of small blocks. The accumulated seek time of numerous non-contiguous reads in HDDs will be significant [30]. In SSDs, reducing the number of non-contiguous reads is also preferred for improving performance [21].

Limitation #2 (Slow degraded read for small blobs under contiguous layout). Contiguous layout, which merges multiple small blobs into the equal-sized large blocks, is a common way to achieve better access and I/O efficiency in erasure-coding-based storage systems (e.g., Facebook f4 [29]). However, Clay codes repair data with the granularity of a block [44] due to their high sub-packetization level, while the degraded read may read a small size blob, such that they may degraded read unnecessary data in the contiguous layout which leads to read amplification.

Impact of limitations. We evaluate the repair performance on a local machine equipped with a 8v Intel i7-11700 CPU at 2.5GHz, 16GB RAM and 1TB 5900RPM HDD. For limitation #1, we generate varying sizes of blobs and encode them with Clay codes under the same set in §5. Figure 3 (a) shows the single-block repair performance for various-size blobs encoded by different (n, k) Clay codes under stripe layout, the repair speed of Clay codes for small blobs is commonly slow and the repair performance drops dramatically as the blob size further decreases from 1 MB to 10 KB. For limitation #2, we set varying block size (e.g., 4 MB, 8 MB and 16 MB) to merge small blobs into a block, and then degraded read different size of small blobs within the block. Figure 3 (b) shows the degraded read performance of (6, 4) Clay code drops dramatically as the blob size decreases on the merging methods (e.g., degraded reading a 1024 KB blob in a 16 MB merged block is 74% slower than degraded reading a 4096 KB blob in a 16 MB merged block). Also, overall degraded read performance reduces significantly as the merge size increases (e.g., degraded reading a 2048 KB blob in a 16 MB merged block is 75% slower than in a 4 MB merged block).

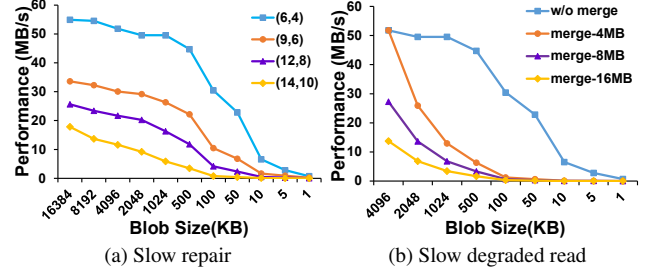


Figure 3: Impact of limitations for Clay codes.

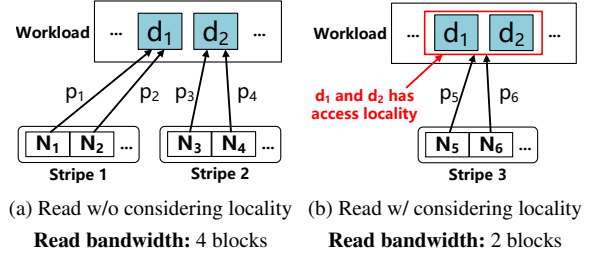


Figure 4: Motivating example.

2.4 Revisiting Non-Systematic MSR Codes

From §2.3, we find that the root cause of limitations for Clay codes in handling small blobs is their high sub-packetization level ($\alpha = (n-k)^{\lceil \frac{n}{n-k} \rceil}$). Fundamentally, the high sub-packetization level is to allow state-of-the-art MSR to ensure access-optimality while maintaining the systematic form [46], so relaxing either the property “access-optimal” or “systematic” can be helpful to reduce the sub-packetization level.

Potential of non-systematic MSR codes. Traditional MSR codes [15] are designed to be “non-systematic”, thus they only requires each block to be divided into $n-k$ sub-blocks (i.e., the lowest sub-packetization level), while each helper has to read and encode its *entire* data into a sub-block transferred to the requestor to repair the lost block, which is not the original one but keeps the MDS property (called *functionally*), without ensuring “access-optimal” properties; F-MSR codes [14] (§2.2) achieve “access-optimal” property with the lowest sub-packetization level, which is more friendly to small blobs than Clay codes. This motivates us to revisit non-systematic MSR codes to improve the repair performance for small blobs.

Challenge of non-systematic MSR codes. Despite the lowest sub-packetization level of non-systematic MSR codes, the major challenge is that their read performance is significantly impaired by their non-systematic form. Reading one data block has to decode k blocks from other nodes, thus leading to *read amplification*. For example, in Figure 1(b), to read a data block D_1 , non-systematic MSR codes have to read four sub-blocks $p_{1,1}, p_{1,2}$ (from N_1) and $p_{2,1}, p_{2,2}$ (from N_2) to decode D_1 , and the *read bandwidth* (i.e., the transferred blocks to read the requested block) is $4/2 = 2$ blocks (due to $\alpha = 2$). Therefore, our main goal is to reduce read amplification of non-systematic MSR codes such that we can leverage them on small blobs of blob storage systems.

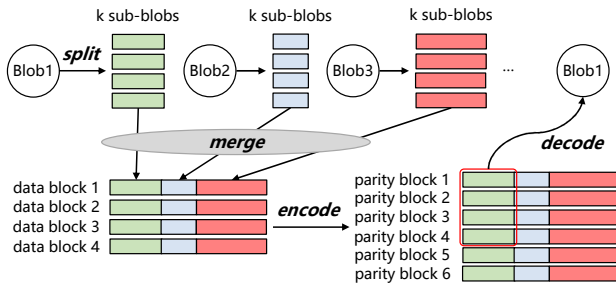


Figure 5: Split-merge-encode scheme.

3 Design

We design NCBlob with the following goals:

- **Reduced read amplification:** NCBlob designs two encoding schemes based on two types of access localities of blobs to reduce the read amplification of non-systematic MSR codes in blob storage (§3.1).
- **Efficient repair:** NCBlob takes a hybrid of non-systematic MSR codes and Clay codes to achieve efficient repair via performing the former on small blobs and the latter on large blobs (§3.2).
- **Generalized parameters:** NCBlob proposes a rotation-based sub-block selection scheme which keeps the MDS property after tens of thousands of iterative repairs when $3 \leq n - k \leq 4$ via simulations, and ensures that the MDS property is always kept when $n - k = 3$ by proof (§3.3).

3.1 Locality-based encoding for small blobs

Insight. To accelerate read of non-systematic MSR codes, our insight is that storage systems often leverage *data access locality* to optimize access performance. The principle of access locality says that if we prefetch a group of data with access locality from disks to memory, then when any bytes from that group of data is accessed in memory, there is a good chance that other data in the group will also be accessed.

For example, Facebook manages data in μ -shards, each of which contains data elements that exhibit data access locality with client applications, to boost read performance [7]. Similarly, deduplication storage systems group deduplicated blocks in containers, each of which exhibits data access locality among blocks, to improve restore performance [22].

Design idea. The insight motivates us to accelerate read of non-systematic MSR codes by performing encoding on the group of data with data *read* access locality. Consider a group of data that has read access locality, we divide the data into k data blocks and then non-systematically encode them into n parity blocks across n nodes. In this way, even if reading one data block in the group of data has to read k parity blocks to reconstruct all k data blocks, all the $k - 1$ remaining reconstructed data blocks (excluding the currently read one) will also be read immediately due to read access locality within the group, thus eliminating read amplification.

Figure 4 illustrates our design idea. Consider two data blocks d_1 and d_2 which have read access locality. In Figure

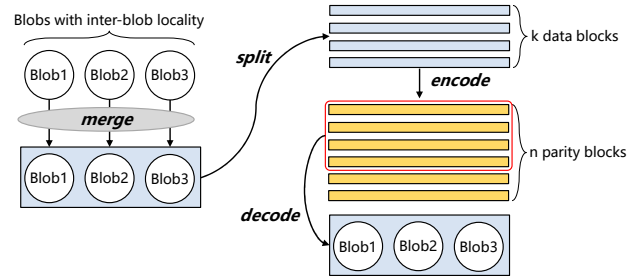


Figure 6: Merge-split-encode scheme.

4(a) without considering the locality, a (4,2) non-systematic MSR codes may encode d_1 and d_2 in two different stripes *Stripe 1* and *Stripe 2*, so reading d_1 and d_2 has to retrieve four blocks p_1, p_2, p_3 and p_4 from four nodes N_1, N_2, N_3 and N_4 , respectively. In contrast, in Figure 4(b) with considering the locality, if we encode d_1 and d_2 together into one stripe *Stripe 3*, then reading d_1 and d_2 only needs two blocks p_5 and p_6 from two nodes N_5 and N_6 , respectively.

Design details. Based on the above design idea, we first give two types of read access localities existing in small blobs: (i) *intra-blob locality* and (ii) *inter-blob locality*.

- **Intra-blob locality:** Some small blob itself is commonly accessed as a whole (e.g., a photo), so that if any byte from such a blob is read, the rest of it should be pre-fetched [41]. Thus, we can directly perform coding on the blob.
- **Inter-blob locality:** Multiple some blobs are often accessed together [18]. Specifically, these blobs often share the same attributes in their metadata (e.g., user ID, app ID, or region in [27]). Thus, we can group them together and perform coding on the group of these blobs.

Second, we recall that stripe layout that divides every blob cannot match small blobs, while contiguous layout that merge small blobs is more friendly for small blobs (§2.3). Thus, we consider how to merge small blobs with the help of the above two read access localities, and design two corresponding non-systematic encoding schemes as follows.

(1) **Split-merge-encode scheme.** Based on intra-blob locality, our proposed encoding scheme is to first split each of the small blobs before performing merging and encoding. Specifically, NCBlob has three steps: (Step 1) We first split one small blob into k equal-sized sub-blobs. (Step 2) We then merge the k sub-blobs of multiple blobs into k data blocks. (Step 3) We finally encode k data blocks into n parity blocks via (n, k) non-systematic MSR codes. In this way, reading any blob can be decoded from just the same amount of bytes as the blob from the parity blocks without read amplification.

As illustrated in Figure 5, each of three blobs (i.e., Blobs1, 2, 3) is first split into $k = 4$ sub-blobs and then merged into data blocks (e.g., the first data block 1 is formed by merging the first sub-blob of all the three blobs). Finally, the four data blocks are encoded into six parity blocks with (6,4) non-systematic MSR codes. Note that reading Blob1 requires reading only the first segment of bytes from each of four par-

ity blocks to decode Blob1, where the size of segment is equal to that of sub-blob of Blob1, such that reading Blob1 only transfers the size of Blob1, incurring no read amplification.

(2) *Merge-split-encode scheme.* Although the split-merge-encode scheme successfully reduces read amplification of small blobs in NCBlob, reading a large number of small blobs still incurs significant I/O overhead, which degrades the read performance. Fortunately, we can leverage the inter-blob locality to further accelerate read performance of NCBlob, and our proposed encoding scheme is to first merge multiple small blobs that have locality before performing splitting and encoding. Specifically, NCBlob has three steps: (Step 1) We first merge multiple small blobs with inter-blob locality into a fixed-size (e.g., 4 MB by default) group. (Step 2) We then split the group into k data blocks. (Step 3) We finally encode the k data blocks into n parity blocks via (n, k) non-systematic MSR codes. In this way, when any blob of the group is read, NCBlob needs to read k out of n parity blocks from disks to reconstruct all the blobs of the group in DRAM, but excluding the currently read blob, all the remaining blobs will also be read immediately due to the inter-blob locality. This ensures that the remaining blobs can be directly read from DRAM instead of disks, which reduces the read amplification.

As illustrated in Figure 6, Blob1, Blob2 and Blob3 that have inter-blob locality are first merged into a group, and then split into k data blocks and finally encoded into n parity blocks. Suppose we first read Blob1, and then NCBlob has to read k parity blocks whose size equals to the size of Blob1, Blob2 and Blob3, and also reconstructs Blob2 and Blob3 of the group, but the upcoming reads of Blob2 and Blob3 can be handled from DRAM, without additional reads from disks.

Note. We first adopt the merge-split-encode scheme for multiple small blobs with inter-blob locality, and then adopt the split-merge-encode scheme for the remaining small blobs with intra-blob locality. Many systems offer locality hints to help decide merge size and blob grouping way. For instance, application developers can provide locality hints [41], and f4 [29] can obtain warm blobs' locality hints based on Haystack's hot version workloads [11]. If no hints are given initially, we can first apply intra-blob locality to all small blobs (as each is often accessed entirely) [18], then analyze workloads after the system runs for a period of time to obtain and apply the inter-blob locality to co-accessed blobs.

3.2 Hybrid MSR codes for all blobs

Challenge. As analyzed in §2.4, non-systematic MSR codes that have the lowest sub-packetization level are promising to repair small blobs efficiently, but it remains unknown how to efficiently repair large blobs which are not negligible in blob storage [44].

Design idea. Based on observations in §2.3, Clay codes still perform well on large blobs. This motivates us to design a hybrid encoding architecture, which encodes small blobs with

non-systematic MSR codes, and encodes large blobs with Clay codes, so as to ensure efficient repair for both small and large blobs. Further, to decide the blob size that differentiates small and large blobs, we first characterize Clay codes' single-block repair time, and then analyze how to select the threshold of blob size for differentiating small and large blobs, which enables us to employ the corresponding codes. We specify the above two steps as follows.

Characterizing the single-block repair time. We consider the repair for the (n, k) Clay code in a cluster with the network bandwidth w . Let the blob size be B and the block size be B/k since each large blob is split into k data blocks and encoded with (n, k) Clay code. The single-block repair time of Clay codes can be obtained by $T = T_N + T_D + T_S + T_C$, where T_N is the data transfer time over network, T_D is the disk read time, T_S is the disk seek time and T_C is the computation time.

For the data transfer time, each node of $n - 1$ helpers in the (n, k) Clay code transfers $1/(n - k)$ blocks to the requestor, and the repair bandwidth is $(n - 1)B/(n - k)k$ and thus the data transfer time is $T_N = \frac{(n-1)B}{(n-k)kw}$. For the disk read and seek time, we assume that the I/O throughput of reading data sequentially from the disk is θ and the average disk seeking time is s , then we can obtain $T_D = \frac{B}{(n-k)k\theta}$ and $T_S = \lambda s$, where λ is the number of I/Os. Finally, the single-block repair time is $T = \frac{(n-1)B}{(n-k)kw} + \frac{B}{(n-k)k\theta} + \lambda s + T_C$.

Selecting the blob size threshold for differentiating blobs.

Based on Clay codes' single-block repair time T characterized above, we find that T is linearly increased with the blob size B for a determined (n, k) Clay code in the specific cluster. When B is small (e.g. hundreds of KB), we find that the disk seeking time $T_S = \lambda s$ dominates T . For example, for a 1 MB blob encoded with $(14, 10)$ Clay code, the repair bandwidth is $\frac{(n-1)B}{(n-k)k} = \frac{13 \times 1 \text{ MB}}{4 \times 10} \approx 333 \text{ KB}$. Consider a cluster that all nodes are connected with a 20Gbps network and each node is equipped with 7200RPM HDD, the data transfer time will be $T_N \approx 0.13 \text{ ms}$; yet even the seeking time of one I/O can be more than 10 ms [32], which makes T_S more than tens or even hundreds of ms; each node reads about 26 KB of data and all nodes can perform reads in parallel, so the disk read time $T_D \approx 0.17 \text{ ms}$ under an average I/O throughput of 7200RPM HDD $\theta = 150 \text{ MB/s}$; the computation speed of repairing a lost block in Clay codes can achieve several GB/s [48], which make the computation time T_C less than the data transfer time. Thus, when the blob is small (less than 1 MB), its disk seeking time T_S dominates the repair time T .

We adjust the blob size from large to small until T of the corresponding blob size begins to be dominated by T_S . For the blobs encoded by Clay codes, T drops linearly as the blob size decreases until T_S dominates T , which means the single-block repair performance (block size/repair time) keeps constant until T_S dominates T . Thus, NCBlob determines the threshold just when T_S dominates T , then the blobs that are larger than the threshold can apply Clay codes for a constant single-block

repair performance. If we continue to perform Clay codes on the blobs that are smaller than the threshold, then they will face the high sub-packetization problem of Clay codes (see §2.3) as T_S begins to dominate T . Thus, NCBlob chooses to use non-systematic MSR codes for the blobs that are smaller than the threshold, such that they can have decent single-block repair performance due to the low sub-packetization level. In this way, we can differentiate small blobs (not larger than the threshold) and large blobs (larger than the threshold).

In NCBlob, we encode small blobs with non-systematic MSR based on the intra-blob and inter-blob localities (§3.1), and encode large blobs with Clay codes via following state-of-the-art practices [26, 29, 44] to optimize the repair performance. In this way, small blobs can be efficiently repaired by non-systematic MSR codes that have the lowest sub-packetization level while maintaining good read performance via the locality-based encoding; large blobs can be efficiently repaired by Clay codes' practices while their systematic property enables good read performance as RS codes.

3.3 Repair for Generalized Coding Parameters

Challenge. Existing non-systematic MSR codes with access-optimality only support $n - k = 2$ (i.e., F-MSR codes [14]), while Clay codes can tolerate any $n - k$ failures with common coding parameters. Thus, we need to generalize non-systematic MSR codes to support more common parameters.

Non-systematic encoding. NCBlob performs non-systematic MSR codes on the k data blocks generated from the locality-based encoding scheme (see §3.1) as follows: (1) NCBlob first divides each data block D_i ($1 \leq i \leq k$) into $\alpha = n - k$ sub-blocks $d_{i,1}, d_{i,2}, \dots, d_{i,n-k}$. (2) NCBlob then encodes the $k(n - k)$ original data sub-blocks into n parity blocks, where each parity block P_j ($1 \leq j \leq n$) contains $\alpha = n - k$ sub-blocks $p_{j,1}, p_{j,2}, \dots, p_{j,n-k}$ with an encoding matrix EM , which is an $n(n - k) \times k(n - k)$ RS based encoding matrix with the coding coefficients $(a_{l,m})_{l=1,2,\dots,n(n-k), m=1,2,\dots,k(n-k)}$ via Cauchy Matrix in the Galois Field $GF(2^8)$ or $GF(2^{16})$. (3) NCBlob finally distributes n parity blocks $P_1, \dots, P_n$ across n nodes $N_1, \dots, N_n$, respectively. Clearly, NCBlob ensures the lowest sub-packetization (i.e., $\alpha = n - k$), while achieving storage-optimality based on Cauchy Matrix based encoding.

Repair-optimal and access-optimal repair. To repair a lost block, NCBlob selects one sub-block from each of $n - 1$ helpers transferred to requestor. This ensures NCBlob is repair-optimal and access-optimal (see §2.2), as both the repair bandwidth and the amount of I/Os are $n - 1$ sub-blocks (i.e., $\frac{n-1}{n-k}$ blocks) that are minimum [15]. Here, the difficulty is to select which sub-block each helper needs to transfer.

Rotation-based selection. NCBlob needs to select one sub-block from each helper in the r^{th} repair. To keep repair MDS property, F-MSR codes make the sub-block selected in the r^{th} repair *different* [14, 19] from the sub-block selected in the $(r - 1)^{th}$ repair, which makes it more likely to obtain the new

block that holds the MDS property after the $(r + 1)^{th}$ repair.

NCBlob extends this idea by asking each helper to select the sub-block *in rotation* for each iterative repair. For example, we suppose that the $(r - 1)^{th}$ repair for a lost block (say P_0) is successful and NCBlob performs the r^{th} repair for a lost block (say P_1) by the two-phase checking heuristic. If NCBlob repairs P_0 by selecting the s^{th} sub-block of each helper, then it will repair P_1 by selecting the $((s + 1) \bmod (n - k))^{th}$ sub-block of each helper. Note that the rotation ensures that the selected sub-blocks for the current repair iteration are least-recently-used to make iterative repairs the most sustainable.

Two-phase Checking. Non-systematic MSR codes reconstruct a lost block functionally during repair; the repaired block is not exactly the lost one, but the MDS property needs to hold after iterative repairs. Given that the $(r - 1)^{th}$ repair for a lost block is successful, NCBlob performs the r^{th} repair for a lost block based on a *two-phase* checking heuristic first proposed by F-MSR codes [14]: NCBlob first checks if the MDS property holds after the r^{th} repair, then checks if the MDS property still holds after the $(r + 1)^{th}$ repair for any possible block failure (called *repair MDS property* [19]).

Specifically, after rotation-based selecting the $n - 1$ transferred sub-blocks from $n - 1$ helpers, the requestor encodes them into $n - k$ sub-blocks to repair the lost block with an $(n - k) \times (n - 1)$ coefficient matrix, called *repair matrix*, denoted by RM , where each element is randomly selected in Galois Field $GF(2^8)$ or $GF(2^{16})$. Then we check if the MDS and repair MDS properties are still met, and if any property check fails, we return to regenerate another RM for the encoding matrices. Note that we only handle the encoding coefficients here, so the checking overhead does not depend on the block size. If the check succeeds, the requestor retrieves the selected sub-blocks and encodes them with the checked repair matrix into the repaired block.

Simulation test. Note that the key difference between F-MSR codes (only support $n - k = 2$) and NCBlob is that NCBlob's rotation-based selection aims to tolerate more than two failures. Thus, to verify if our proposed selection method works for generalized coding parameters, we conduct simulations (open-sourced with our prototype system) on some common coding parameters ($n - k = 3$ or 4) to show how many repair iterations NCBlob keeps the MDS and repair MDS properties. After our simulation test, we find that for common coding parameters in practice (e.g., $(n, k) = (9, 6)$ [33] and $(14, 10)$ [29]), NCBlob can still work after more than tens of thousands of repair iterations in $GF(2^{16})$ (e.g., 800K times for $(9, 6)$ NCBlob and 10K times for $(14, 10)$ NCBlob). Considering a node failure with an average lifetime of 4 years [42], there will expect to be a node failure in a $(14, 10)$ stripe about every 3.4 months (i.e., 48 months/14). Assuming that NCBlob repairs a node failure each time, it can sustain a lifetime of about $\frac{48 \times 10000}{14 \times 12} \approx 2857$ years for $(14, 10)$ NCBlob works after 10K repair iterations, which shows our rotation-based selection works for generalized coding parameters.

Proof for NCBlob. In addition, we prove that NCBlob works for any coding parameters with $n - k = 3$ by Theorem 1 (its detailed proof is in supplemental material file).

Theorem 1. $(k + 3, k)$ NCBlob always exists.

Sketch of Proof. The idea is to first add a new property *evolved repair MDS* (*erMDS*, first proposed in [19]) property that characterizes more conditions of iterative repairs than the repair MDS property, and then prove that the MDS, repair MDS and erMDS properties are met after the r^{th} repair, provided that they have been met before the r^{th} repair, formulated as Lemmas 1, 2 and 3, respectively, as follows. $\square$

Lemma 1. NCBlob meets the MDS property after the r^{th} repair, if the repair MDS property is met before the r^{th} repair.

Lemma 2. NCBlob meets the repair MDS property after the r^{th} repair, if the erMDS property is met before the r^{th} repair.

Lemma 3. NCBlob meets the erMDS property after the r^{th} repair, if the erMDS property is met before the r^{th} repair.

Sketch of Proof. Lemma 1 has been proved in [19]. We prove Lemmas 2 and 3 by induction on r . Assuming that any $3k$ sub-blocks of NCBlob can decode all k data blocks (called *decodable*) when $r = 0$ (induction hypothesis). Suppose NCBlob is decodable after the r^{th} repair, and consider its $(r + 1)^{th}$ repair. For Lemma 2, we construct the repair MDS property checking set of sub-blocks that includes $3k$ sub-blocks from any three nodes (each provides one sub-block) and any other $(k - 1)$ nodes (each provides three sub-blocks), and prove that any of those $3k$ sub-blocks are decodable after the $(r + 1)^{th}$ repair. For Lemma 3, we construct the erMDS property checking set of sub-blocks that includes $3k$ sub-blocks from any three nodes (randomly select six sub-blocks out of three nodes, at least one sub-block per node) and any other $(k - 2)$ nodes (each provides three sub-blocks), and prove that any of those $3k$ sub-blocks are decodable after the $(r + 1)^{th}$ repair. $\square$

3.4 Discussion

NCBlob in warm storage. Despite reduced read amplification, NCBlob’s reads bring additional computation overhead, which decreases read performance a little (§5.2 shows the results: 2.1% to 10.5%). Nevertheless, warm storage is for accessed infrequently, so we claim that NCBlob is suitable for warm storage, which often allows small read performance loss than hot storage (e.g., 17.6% read performance loss in Facebook: 17ms for warm storage and 14ms for hot storage [29]).

Another hybrid coding scheme. GeoPartition [44] also propose a hybrid coding scheme by encoding small blobs with RS codes and large blobs with Clay codes. However, small blobs dominate the blob storage, so GeoPartition’s RS coded small blobs cannot be repaired efficiently. We will evaluate GeoPartition’s hybrid coding scheme in the evaluation §5.

Parameter setting. We consider $n - k \leq 4$ for NCBlob, which is a typical setting for warm storage systems (e.g., Facebook’s f4 uses (14, 10) coding [29]) and modern cold storage systems

(e.g., Azure’s Pelican uses (18, 15) coding [10]). If $n - k$ becomes larger ($5 \leq n - k \leq 8$), NCBlob is more likely to ensure the MDS/repair MDS/erMDS properties (see §3.3) as the rotation-based selection (see §3.3) on a larger set of $n - k$ sub-blocks has more potentials to sustain iterative repairs.

Assess patterns lack locality. If access patterns lack locality, we can use systematic MSR codes (e.g., Clay codes) for small blobs to ensure good read performance. Nevertheless, in many blob storage systems, locality is widely present in object-based storage [18] and public cloud [41]. Intra-blob locality is especially prevalent in blob storage as many small blobs are often entirely accessed (e.g., photos [11, 29]).

Multiple failures. Our work focuses on single failures as they are more common than multiple failures [20]. If a stripe has multiple failed blocks, we resort to conventional repair by reading sufficient available blocks as in prior work [17, 25].

4 Implementation

Architecture. We implement NCBlob in C++ on Linux with about 18.6 K SLoC. NCBlob comprises three components: Client, MetadataServer and DataServer. Clients generate I/O requests that store or read blobs. MetadataServer stores block metadata and manages DataServers, including a Coordinator that coordinates repair and read operations via generating and assigning tasks among DataServers. Each DataServer runs an Agent to perform the assigned tasks and a Requestor that sends repair requests during repair. We use a key-value store in DataServers to store and manage blobs, where each key is the unique index to track the corresponding block. Each DataServer is bound to a specific disk and multiple processes of DataServers can run on the same physical machine with multiple disks. NCBlob uses Redis [4] for internal communications among the MetadataServer and the DataServers.

Locality management. The locality management module runs on clients to manage small blobs based on their specific access locality. We implement two approaches to manage small blobs: (i) for intra-blob locality, each small blob is by default considered as a data group; (ii) for blobs with inter-blob locality, clients merge them into a fixed-sized data group.

Read parallelization. We implement a *slice-grained parallelized read* (SPR for short) scheme to enhance the read performance of non-systematic MSR codes. We borrow the idea of “slice” in repair parallelization [25] to divide each sub-block into the fine-grained slices. Since both a repair and a read in non-systematic codes are basically a decoding process, decoding a block can be parallelized at the slice level, such that we can make each node provide the same bandwidth for reads and all nodes’ bandwidth can be evenly and fully utilized.

As illustrated in Figure 7 (a), a (5, 3) NCBlob has five nodes $\{N_1, N_2, N_3, N_4, N_5\}$, and each node stores a block consists of $n - k = 2$ sub-blocks. SPR first divides each sub-block into $n = 5$ slices, and then we collect every $k(n - k) = 6$

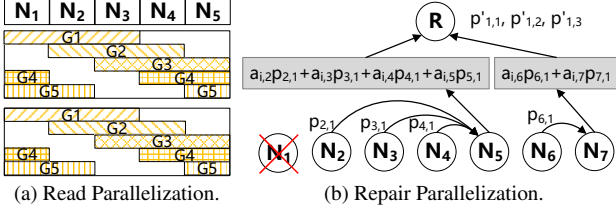


Figure 7: Examples of read and repair parallelization.

slices from all the $n - k = 2$ sub-blocks of the $k = 3$ nodes in a round-robin way to form group G_i ($1 \leq i \leq n$) (e.g., group G_1 contains 6 slices from all 6 sub-blocks of 3 nodes $\{N_1, N_2, N_3\}$). Decoding each group can get the original data of the same offset in the $k = 3$ data blocks (e.g., G_1 can decode the first slice of each of the six sub-blocks), since content with the same offset of a stripe will be encoded or decoded together in bytes [25]. In this way, all five nodes (each provides six slices from three groups) can be evenly used in parallel reads.

Single-block repair parallelization. NCBlob implements a *group-based parallel repair* (GPR for short) scheme based on the concept of partial-parallel-repair [28] (PPR) to decompose a repair operation of the non-systematic MSR codes into parallel partial sub-operations.

Figure 7(b) illustrates a $(7, 4)$ NCBlob with GPR. Repairing a lost block can be decomposed into repairing three sub-blocks $p'_{1,1}, p'_{1,2}$ and $p'_{1,3}$. We first divide all six helpers into two groups. For each group, we choose a gatherer to gather the selected sub-blocks for repair (e.g., N_5 and N_7 are the gatherers). We then select the helpers responsible for decoding the lost sub-blocks that will be repaired functionally. For example, N_5 gathers $\{p_{2,1}, p_{3,1}, p_{4,1}, p_{5,1}\}$ which can provide the linear combinational parts of the repaired sub-blocks (i.e., $a_{1,2}p_{2,1} + a_{1,3}p_{3,1} + a_{1,4}p_{4,1} + a_{1,5}p_{5,1}$), and N_7 gathers $\{p_{6,1}, p_{7,1}\}$ which can provide $a_{1,6}p_{6,1} + a_{1,7}p_{7,1}$. We finally send the required sub-blocks $p'_{1,1}, p'_{1,2}$ and $p'_{1,3}$ to the requestor R to repair the lost block, where $p'_{1,i} = a_{1,2}p_{2,1} + a_{1,3}p_{3,1} + \dots + a_{1,7}p_{7,1}$. In this way, we can use N_5 and N_7 to transfer the partial linear combinations of each sub-block to repair three sub-blocks $p'_{1,1}, p'_{1,2}$ and $p'_{1,3}$ in parallel.

Coding module. We implement RS and Clay codes based on the erasure coding library of Ceph [3], and traditional non-systematic MSR codes (i.e., random linear network codes) is implemented based on Jerasure. We further implement parallelism in NCBlob based on ParaRC [26] which has supported the repair parallel schemes of RS codes and Clay codes. The metadata of each stripe (e.g., stripe ID, block ID) is stored in MetadataServer for repair and read operations.

The coding module also implements the threshold calculation for distinguishing large and small blobs as in §3.2. For large blobs, we divide each into k blocks [3, 8, 33] and encode them into a (n, k) stripe via Clay codes [48]. For small blobs, we call the *locality management module* to group small blobs with different access locality and then perform coding on the blob group based on its locality to generate the coded stripes.

Distribution module. This module uses a similar concept

of Placement Group (PG) as in Ceph [3] to evenly distribute blocks over multiple nodes. Each PG is set a fixed (n, k) and the blocks from the same stripe are mapped into the specific PG based on the defined coding scheme (e.g., for a PG of $(14, 10)$ Clay code, it contains a group of N nodes ($N > 14$)). PG maps 14 blocks of a $(14, 10)$ stripe onto 14 out of N nodes, with each block mapped to a unique node. For each block, the node with the least capacity consumed for that PG is selected to store the block. MetadataServer stores the indices of PGs and assigns tasks to PGs to repair multiple block failures.

Repair module. To repair a lost block from a Requestor, the Coordinator (i) triggers the repair of $n - 1$ Agents that serve as the helpers, (ii) asks each helper to select one sub-block via rotation-based selection (see §3.3) which is then sent to another helper determined by the GRP scheme, and (iii) sends the encoding coefficients to the helpers, generated by two-phase checking. Finally, the Requestor repairs the lost block functionally. For the decoding operation, the Coordinator sends the metadata to the Requestor and asks all Agents to send slices determined by the SPR scheme to Requestor that decodes the requested block.

5 Evaluation

5.1 Experimental Setup

We evaluate NCBlob via cloud experiments to address two questions: (i) Can NCBlob provide good read performance using locality? (ii) Can NCBlob improve the repair performance via hybrid systematic and non-systematic MSR codes?

Platform. We conduct experiments on Alibaba Cloud [1] with a cluster of 22 nodes, including one Coordinator, 20 Agents and one Client. The Coordinator and Agents are deployed in `ecs.d3s.2xlarge` instances with 8 vCPUs, 32 GB RAM and a 44 TB HDD. The Client runs on an `ecs.u1-c1m4.xlarge` instance with 4 vCPUs and a 16 GB RAM. All instances are connected via a 15 Gbps network. We conduct experiments on RS codes, Clay codes and NCBlob under four practical (n, k) coding parameters: $(6, 4)$, $(9, 6)$ [33], $(12, 8)$ [24] and $(14, 10)$ [29]. We report average results over ten runs and provide the variance of these results.

Workloads. We conduct experiments based on Azure Blob Access Trace [9], a sample of blob accesses in Microsoft's Azure. It contains accesses to 20.3 million blobs of sizes from 8 Bytes to 1.78 GB. We evaluate NCBlob on a sampled workload from the trace of about 60 GB. The ratio of the number of large to small blobs is set to 5%:95%, while the ratio of the total sizes of large to small blobs is set to 4:1, consistent with characteristics observed in the original trace.

Configurations. We compare four coding schemes: (i) encode all blobs with RS codes (RS); (ii) encode all blobs with Clay codes (Clay); (iii) encode large blobs with Clay codes and small blobs with RS codes (Hybrid:RS+Clay, see §3.4) and (iv) NCBlob in §3.2. For single-block repairs, we compare the pipeline repair of RS, Clay and NCBlob: for RS and Clay, we

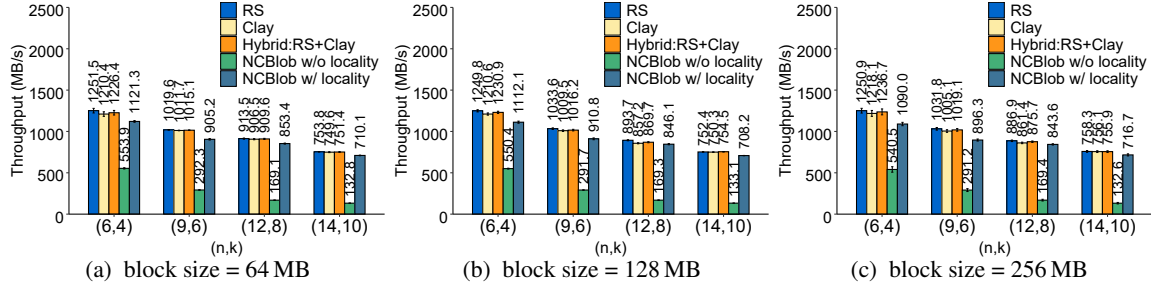


Figure 8: Experiment 1: Read throughput of different coding scheme with varying block sizes for different parameters (n, k) .

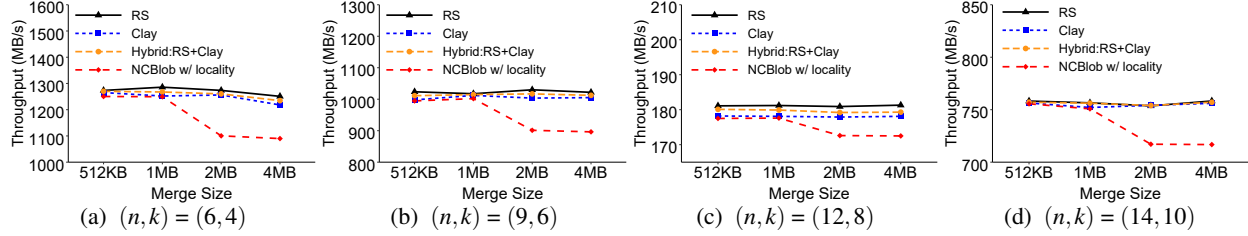


Figure 9: Experiment 2: Varying merge size for the read performance of different parameters (n, k) .

use their state-of-the-art pipelined repair schemes (ECPipe [25] and ParaRC [26]); for NCBlob, we use NCBlob with GRP (see §4). Note that we do not consider F-MSR here as it only supports $n - k = 2$ [14]. For single-blob degraded reads, we compare ECPipe, ParaRC, and NCBlob with SPR. For full-node repair, we compare the centralized repair of RS, Clay and NCBlob, since ParaRC may incur significant computation overhead to find the optimal way to repair a node [26].

5.2 Experiments

Experiment 1 (Read performance). We evaluate the read throughput for RS, Clay, Hybrid:RS+Clay, NCBlob without locality and NCBlob with locality for different block sizes under different (n, k) . After encoding, we perform sequential read operation to read blobs based on the read workloads. NCBlob with locality are encoded using the 1 MB merged groups constructed based on the trace’s access locality.

Figure 8 shows that NCBlob with locality significantly improves the read throughput compared to NCBlob without locality and achieves comparable read throughput with RS codes and Clay codes across all block sizes. For example, in Figure 8 (c), the read throughput of $(12, 8)$ NCBlob with locality is 843.6 MB/s, which is only 2.1% lower than that of $(12, 8)$ Clay codes and 4.9× higher than that of NCBlob without locality. Also, compared to Clay codes, NCBlob with locality has 10.4%, 9.8% and 5.2% lower read throughput for configurations of $(n, k) = (6, 4)$, $(9, 6)$ and $(14, 10)$, when the block size is 256 MB, respectively.

The difference of the read throughput between NCBlob with locality and Clay codes reaches its maximum when $(n, k) = (9, 6)$ and the block size is 64 MB (as in Figure 8 (a)), where NCBlob with locality is 10.5% slower than Clay codes. Nevertheless, as stated in §3.4, in Facebook, the read performance of its warm storage has a loss of 17.6% compared to that of its hot storage, which implies that NCBlob’s read performance is still acceptable for warm storage.

Note that for all coding schemes, the read performance drops with k . The reason is that NCBlob merges multiple blobs as a fixed-size group, which makes blob reading sends read requests to k nodes to obtain each group, such that a larger k leads to more read requests in the system and thus decreases the read performance, since each read request incurs a basic execution cost for the node to read a data block.

Experiment 2 (Impact of merge size). We evaluate the read throughput versus the merge size for NCBlob with locality. We fix the block size at 256 MB and vary the merge size from 512 KB to 4 MB for different (n, k) . Figure 9 shows that smaller merge size benefits the read throughput more. When the merge size is as small as 1 MB or 512KB, NCBlob with locality have the comparable read throughput with RS, Clay and Hybrid:RS+Clay. The read throughput of NCBlob with locality decreases when the merge size become larger. For example, in Figure 9 (a), when the merge size is increase from 1 MB to 2 MB, the read throughput of NCBlob with locality is decreased from 1249.37 MB/s to 1100.36 MB/s, where the read throughput is reduced by 11.9%. The reason is that in our trace, the size of blobs that have inter-blob locality is not too large, and 1MB is often a good choice for merging.

Experiment 3 (Single-block repair performance). We evaluate the single-block repair time for different coding schemes: RS, Clay and non-systematic MSR code of NCBlob, for varying block size from 64 MB to 256 MB under different (n, k) . Note that we do not compare Hybrid:RS+Clay as we only consider single-block repair within a single coded stripe. Figure 10 shows that NCBlob always outperforms Clay codes in small block size under the same n and k . For example, in Figure 10 (c) where $(n, k) = (14, 10)$, the single-block repair time of NSYS is reduced by 35.7% and 44.1%, compared to that of Clay codes and RS codes, respectively. Also, compared with Clay codes, NCBlob achieves better performance gains under $(14, 10)$ than the other (n, k) parameters, since the

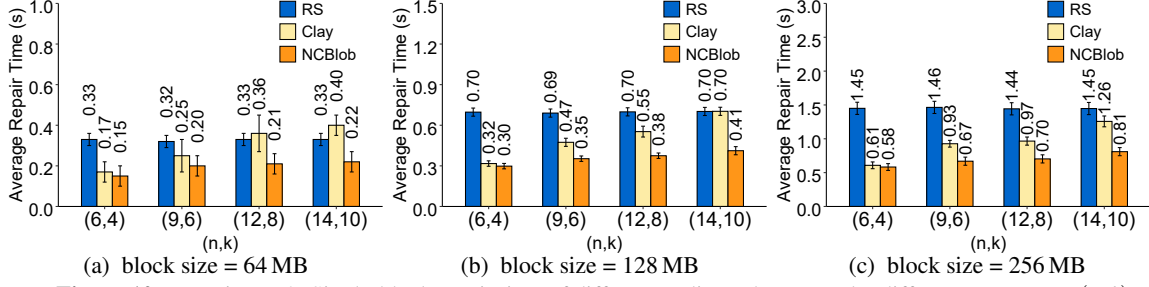


Figure 10: Experiment 3: Single-block repair time of different coding schemes under different parameters (n, k) .

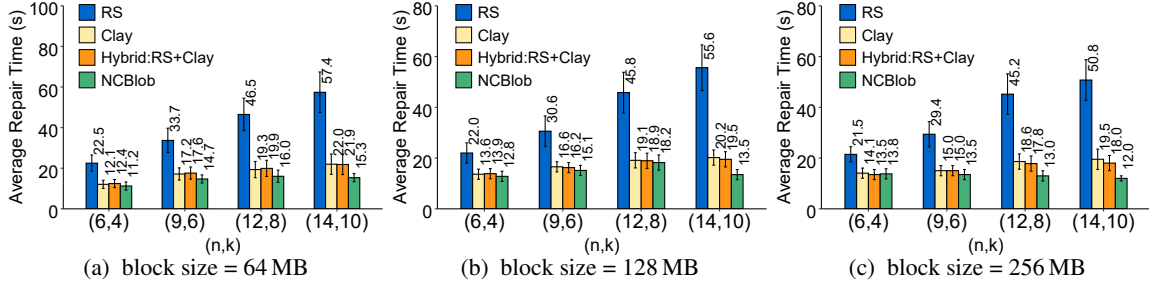


Figure 11: Experiment 4: Full-node repair time of different coding scheme with varying block sizes for different parameters (n, k) .

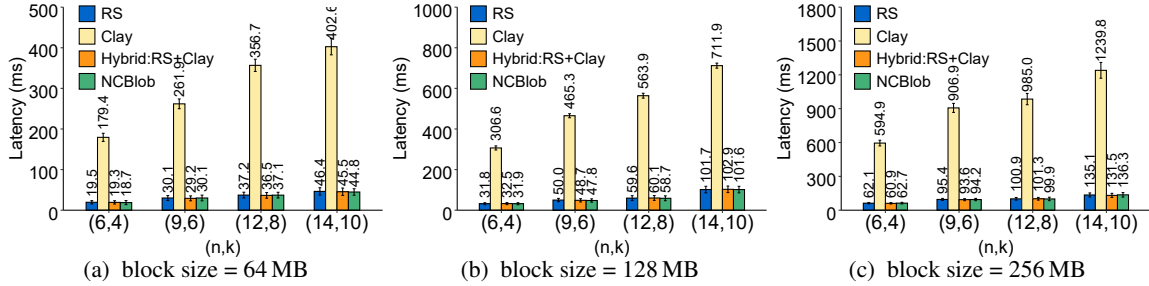


Figure 12: Experiment 5: Single-blob degraded read time of different coding scheme with varying block sizes for different parameters (n, k) .

sub-packetization level of Clay code (exponential with $n - k$) increases much faster than that of NCBlob (always be $n - k$). When the block size is small, the single-block repair performance of Clay codes is significantly reduced. We can see that when the block size is as small as 64 MB, Clay codes is slower than RS with pipeline repair, and NCBlob reduces the single-block repair time by 45.0% compared to Clay codes.

Experiment 4 (Full-node repair performance). We evaluate the full-node repair time in terms of the total repair time of reconstructing a failed node. We first generate the stripes based on the trace of 60GB and distribute the stripes across 20 nodes evenly. Then we turn off one node to trigger the full-node repair and measure the average repair time. We compare RS, Clay, Hybrid:RS+Clay and NCBlob under different (n, k) with different block sizes. Figure 11 shows that, the full-node repair performance of NCBlob outperforms the other repair schemes; the gap of full-node repair performance between NCBlob and Clay/Hybrid:RS+Clay becomes significant when k becomes large. For example, for $(n, k) = (14, 10)$ and the block size is 256 MB, NCBlob reduces the full-node repair time by 33.3%, 38.4% and 76.4% compared with Hybrid:RS+Clay, Clay and RS, respectively. The reason is that Clay codes cannot handle small blobs efficiently due to their exponential sub-packetization level, which confirms the Limi-

tation #1 of Clay codes (see §2.3).

Experiment 5 (Single-blob degraded read performance).

We evaluate the single-blob degraded read performance in terms of the average degraded read latency for different coding schemes, different (n, k) and different block sizes. Figure 12 shows that NCBlob outperforms Clay codes in the single-blob degraded read performance and achieves the comparable degraded read performance with Hybrid:RS+Clay. For example, for $(n, k) = (14, 10)$ and the block size is 256 MB, the degraded read performance of Clay codes falls significantly compared to Hybrid:RS+Clay and our NCBlob, and the degraded read latency of Clay codes is $9.1 \times$ that of our NCBlob. This confirms the Limitation #2 of Clay codes (see §2.3).

Experiment 6 (Encoding performance). We evaluate the encoding performance in terms of the encode throughput for Clay, NCBlob without merge and NCBlob with merge under different (n, k) on the workloads. Figure 13 (a) shows that NCBlob with merge outperforms NCBlob without merge in the encode performance. For $(n, k) = (14, 10)$, the encoding throughput of NCBlob with merge is $2.7 \times$ higher than that of NCBlob without merge. NCBlob with merge also achieves the comparable encoding performance with Clay codes. The encode throughput of (6,4) NCBlob with merge is 104.2 MB/s, which is only 0.5% lower than that of (6,4) Clay codes.

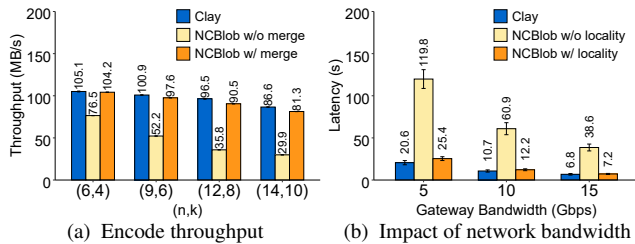


Figure 13: Experiment 6 & Experiment 7: Comparisons of encode throughput of different coding schemes and read performance under different network bandwidth, with 256 MB block size.

Experiment 7 (Impact of network bandwidth). We evaluate the read latency versus the network bandwidth under $(n, k) = (14, 10)$. We vary the network bandwidth from 5 Gbps to 15 Gbps. Figure 13 (b) shows that the read latency of all approaches increases as the network bandwidth decreases, while NCBlob with locality still maintains its advantage.

Experiment 8 (CPU and memory overhead). We evaluate CPU usage and memory overhead for different coding schemes with the block size of 256 MB. Figure 14 (a) shows the CPU usage of NCBlob with locality is lower than that of NCBlob without locality and higher than that of Clay. When $(n, k) = (14, 10)$, NCBlob experiences high CPU usage as it decodes in parallel whereas Clay reads directly. For $(n, k) = (6, 4)$, the CPU usage of NCBlob with locality is 80.1%, which is 4.8% higher than that of Clay. Figure 14 (b) shows that NCBlob with locality can reduce memory overhead compared to NCBlob without locality. For example, the reduction can be up to 56.3% when $(n, k) = (14, 10)$.

6 Related Work

Blob storage. Blob stores are widely used in production for unstructured data management. Microsoft’s Azure [13] implements access tiers to manage blobs with different access patterns and cost considerations [27]. It uses locally repairable codes to protect rarely accessed (i.e., cold) blobs for fault tolerance [20]. Flat Datacenter Storage [30] adopts a scale-out architecture that exposes the available bandwidth across all nodes for high-performance blob accesses. Facebook’s Haystack [11] manages hot blobs (mainly photos and videos) with replication for fault tolerance and fast accesses. It moves aged, less frequently accessed (i.e., warm) blobs to f4 [29], which protects blobs with erasure coding for low-cost fault tolerance while maintaining acceptable access performance. Note that f4’s uses $(14, 10)$ RS coding within a data center and RAID-5-like XOR-coding across data centers; it does not consider repair efficiency. Tectonic [35] extends Haystack and f4 for multi-tenant accesses with exabyte-scale storage and improve resource sharing. Twitter’s Blobstore [47] stores photos and other blobs with replication for fault tolerance. LinkedIn’s Ambry [31] supports decentralized blob storage, leveraging replication for low-latency accesses and load balancing.

Regenerating codes. We focus on MSR codes [15], which provably minimize the repair bandwidth subject to the mini-

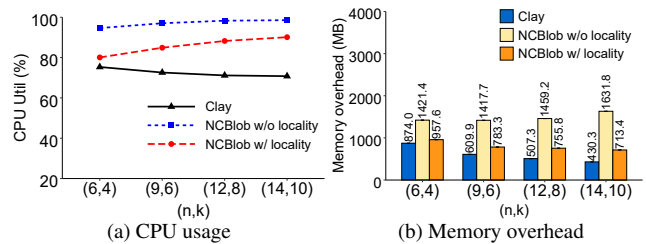


Figure 14: Experiment 8: CPU usage and memory overhead of different coding scheme for different parameters (n, k) .

um storage redundancy based on network coding [5]. Existing MSR codes in the literature are mostly systematic, with representative examples including MISER [43], Zigzag [45], PM-RBT [39], Butterfly [34], HashTag [23], and Clay [48]. Early proposed MSR codes have limited parameter regimes, such as $n \geq 2k - 1$ for PM-RBT codes [39] and $n - k = 2$ for Butterfly codes [34], while Clay codes [48] represent the state-of-the-art systematic MSR codes that support general (n, k) and minimize both repair bandwidth and I/Os. However, Clay codes also have an exponential sub-packetization level with respect to k .

Some studies parallelize a repair of MSR codes to mitigate the I/O overhead. Geometric partitioning [44] divides an MSR-coded object into variable-size blocks to improve both degraded read and node recovery performance. ParaRC [26] divides a repair into sub-operations that are executed in multiple nodes, so as to mitigate the maximum repair load while maintaining the repair efficiency of MSR codes. We show that repair parallelization also applies to NCBlob (§4).

Some studies consider non-systematic MSR codes. F-MSR codes [14] target multi-cloud storage and focus on $n - k = 2$. It is shown that each repair in F-MSR codes still maintains the MDS property [19]. Clustered network coding (CNC) [6] targets data-center storage and is evaluated under larger (n, k) . Both F-MSR codes and CNC target cold archival.

7 Conclusions

Warm blob storage often adopts erasure coding but state-of-the-art ones have practical repair limitations. We propose NCBlob, a locality-based non-systematic MSR codes and a hybrid MSR coding architecture for both efficient reads and repairs in practice under generalized common coding parameters. We implement NCBlob and Cloud experiments demonstrate the efficiency of NCBlob in single-block repairs and full-node repairs with small read throughput loss.

Acknowledgments. We thank our shepherd, Ningfang Mi, and the anonymous reviewers for their comments. This work was supported by National Key Research and Development Program of China (No. 2023YFB4502100), National Natural Science Foundation of China (No. 62272185), Key Research and Development Program of Hubei Province (No. 2021BAA189), and Shenzhen Science and Technology Program (No. JCYJ20220530161006015). The corresponding author is Yuchong Hu.

References

- [1] Alibaba Cloud. <https://www.alibabacloud.com/product/ecs-pricing-list/en>.
- [2] Amazon S3 Glacier & S3 Glacier Deep Archive. <https://aws.amazon.com/glacier/>.
- [3] Ceph - Erasure code. <https://docs.ceph.com/en/latest/rados/operations/erasure-code>.
- [4] Redis. <http://redis.io/>.
- [5] Rudolf Ahlswede, Ning Cai, S-YR Li, and Raymond W Yeung. Network Information Flow. *IEEE Trans. on Information Theory*, 46(4):1204–1216, 2000.
- [6] Fabien André, Anne-Marie Kermarrec, Erwan Le Merer, Nicolas Le Scouarnec, Gilles Straub, and Alexandre Van Kempen. Archiving cold data in warehouses with clustered network coding. In *Proc. of ACM Eurosys*, 2014.
- [7] Muthukaruppan Annamalai, Kaushik Ravichandran, Harish Srinivas, Igor Zinkovsky, Luning Pan, Tony Savor, David Nagle, and Michael Stumm. Sharding the shards: managing datastore locality at scale with akkio. In *Proc. of USENIX OSDI*, 2018.
- [8] Apache Software Foundation. Hadoop HDFS. <http://hadoop.apache.org/>.
- [9] Azure. Azure functions blob dataset. <https://github.com/Azure/AzurePublicDataset>.
- [10] Shobana Balakrishnan, Richard Black, Austin Donnelly, Paul England, Adam Glass, Dave Harper, Sergey Legtchenko, Aaron Ogus, Eric Peterson, and Antony Rowstron. Pelican: A building block for exascale cold data storage. In *Proc. of USENIX OSDI*, 2014.
- [11] Doug Beaver, Sanjeev Kumar, Harry C Li, Jason Sobel, and Peter Vajgel. Finding a needle in haystack: Facebook’s photo storage. In *Proc. of USENIX OSDI*, 2010.
- [12] Oded Berman. Comparing storage tiering options on aws, azure, and google cloud. <https://bluexp.netapp.com/blog/cts-blg-storage-tiering-options-on-aws-azure-and-google-cloud>, 2020. Retrieved in Sep 2024.
- [13] Brad Calder, Ju Wang, Aaron Ogus, Niranjana Nilakantan, Arild Skjolsvold, Sam McKelvie, Yikang Xu, Shashwat Srivastav, Jiesheng Wu, Huseyin Simitci, et al. Windows azure storage: a highly available cloud storage service with strong consistency. In *Proc. of ACM SOSP*, 2011.
- [14] Henry C. H. Chen, Yuchong Hu, Patrick P. C. Lee, and Yang Tang. NCCloud: A network-coding-based storage system in a cloud-of-clouds. *IEEE Trans. on Computers*, 63(1):31–44, 2014.
- [15] Alexandros G Dimakis, P Brighten Godfrey, Yunnan Wu, Martin J Wainwright, and Kannan Ramchandran. Network Coding for Distributed Storage Systems. *IEEE Trans. on Information Theory*, 56(9):4539–4551, 2010.
- [16] Daniel Ford, Francois Labelle, Florentina I. Popovici, Murray Stokel, Van-Anh Truong, Luiz Barroso, Carrie Grimes, and Sean Quinlan. Availability in Globally Distributed Storage Systems. In *Proc. of USENIX OSDI*, 2010.
- [17] Parikshit Gopalan, Cheng Huang, Huseyin Simitci, and Sergey Yekhanin. On the locality of codeword symbols. *IEEE Trans. on Information theory*, 58(11):6925–6934, 2012.
- [18] James Hendricks, Raja R Sambasivan, Shafeeq Sinnamohideen, and Gregory R Ganger. Improving small file performance in object-based storage. Technical report, Carnegie Mellon University, 2006.
- [19] Yuchong Hu, Patrick P. C. Lee, and Kenneth W Shum. Analysis and construction of functional regenerating codes with uncoded repair for distributed storage systems. In *Proc. of IEEE INFOCOM*, 2013.
- [20] Cheng Huang, Huseyin Simitci, Yikang Xu, Aaron Ogus, Brad Calder, Parikshit Gopalan, Jin Li, and Sergey Yekhanin. Erasure Coding in Windows Azure Storage. In *Proc. of USENIX ATC*, 2012.
- [21] Yuhun Jun, Shinhyun Park, Jeong-Uk Kang, Sang-Hoon Kim, and Euseong Seo. We Ain’t Afraid of No File Fragmentation: Causes and Prevention of Its Performance Impact on Modern Flash SSDs. In *Proc. of USENIX FAST*, 2024.
- [22] Iwona Kotlarska, Andrzej Jackowski, Krzysztof Lichota, Michal Welnicki, Cezary Dubnicki, and Konrad Iwanicki. InftyDedup: Scalable and Cost-Effective cloud tiering with deduplication. In *Proc. of USENIX FAST*, 2023.
- [23] Katina Kravlevska, Danilo Gligoroski, Rune E Jensen, and Harald Øverby. Hashtag erasure codes: From theory to practice. *IEEE Transactions on Big Data*, 4(4):516–529, 2018.
- [24] Chunbo Lai, Song Jiang, Liqiong Yang, Shiding Lin, Guangyu Sun, Zhenyu Hou, Can Cui, and Jason Cong. Atlas: Baidu’s key-value storage system for cloud data. In *Proc. of IEEE MSST*, 2015.
- [25] Runhui Li, Xiaolu Li, Patrick P. C. Lee, and Qun Huang. Repair pipelining for erasure-coded storage. In *Proc. of USENIX ATC*, 2017.
- [26] Xiaolu Li, Keyun Cheng, Kaicheng Tang, Patrick P. C. Lee, Yuchong Hu, Dan Feng, Jie Li, and Ting-Yi Wu. ParaRC: Embracing Sub-Packetization for repair parallelization in MSR-Coded storage. In *Proc. of USENIX FAST*, 2023.

- [27] Microsoft. Azure blob storage - access tiers. <https://learn.microsoft.com/en-us/azure/storage/blobs/access-tiers-overview>, Jan 2024.
- [28] Subrata Mitra, Rajesh Panta, Moo-Ryong Ra, and Saurabh Bagchi. Partial-parallel-repair (PPR): a distributed technique for repairing erasure coded storage. In *Proc. of ACM Eurosys*, 2016.
- [29] Subramanian Muralidhar, Wyatt Lloyd, Sabyasachi Roy, Cory Hill, Ernest Lin, Weiwen Liu, Satadru Pan, Shiva Shankar, Viswanath Sivakumar, Linpeng Tang, et al. f4: Facebook’s Warm BLOB Storage System. In *Proc. of USENIX OSDI*, 2014.
- [30] Edmund B Nightingale, Jeremy Elson, Jinliang Fan, Owen Hofmann, Jon Howell, and Yutaka Suzue. Flat datacenter storage. In *Proc. of USENIX OSDI*, 2012.
- [31] Shadi A. Noghabi, Sriram Subramanian, Priyesh Narayanan, Sivabalan Narayanan, Gopalakrishna Holla, Mammad Zadeh, Tianwei Li, Indranil Gupta, and Roy H. Campbell. Ambry: LinkedIn’s scalable geo-distributed object store. In *Proc. of ACM SIGMOD*, 2016.
- [32] Serkay Olmez, Yifan Dai, John Bent, and Remzi Arpacı-Dusseau. Revisiting HDD Rules of Thumb: 1/3 Is Not (Quite) the Average Seek Distance. In *Proc. of IEEE MSST*, 2024.
- [33] Michael Ovsianikov, Silvius Rus, Damian Reeves, Paul Sutter, Sriram Rao, and Jim Kelly. The Quantcast File System. In *Proc. of ACM VLDB*, 2013.
- [34] Lluís Pamies-Juarez, Filip Blagojevic, Robert Mateescu, Cyril Guyot, Eyal En Gad, and Zvonimir Bandić. Opening the chrysalis: On the real repair performance of MSR codes. In *Proc. of USENIX FAST*, 2016.
- [35] Satadru Pan, Theano Stavrinou, Yunqiao Zhang, Atul Sikaria, Pavel Zakharov, Abhinav Sharma, Mike Shuey, Richard Wareing, Monika Gangapuram, Guanglei Cao, et al. Facebook’s tectonic filesystem: Efficiency from exascale. In *Proc. of USENIX FAST*, 2021.
- [36] Dimitris S Papailiopoulos and Alexandros G Dimakis. Locally repairable codes. *IEEE Trans. on Information Theory*, 60(10):5843–5855, 2014.
- [37] James S Plank, Jianqiang Luo, Catherine D Schuman, Lihao Xu, and Zooko Wilcox-O’Hearn. A performance evaluation and examination of open-source erasure coding libraries for storage. In *Proc. of USENIX FAST*, 2009.
- [38] Korlakai Vinayak Rashmi, Nihar B Shah, and P Vijay Kumar. Optimal exact-regenerating codes for distributed storage at the msr and mbr points via a product-matrix construction. *IEEE Transactions on Information Theory*, 57(8):5227–5239, 2011.
- [39] KV Rashmi, Preetum Nakkiran, Jingyan Wang, Nihar B Shah, and Kannan Ramchandran. Having your cake and eating it too: Jointly optimal erasure codes for I/O, storage, and network-bandwidth. In *Proc. of USENIX FAST*, 2015.
- [40] Irving S Reed and Gustave Solomon. Polynomial Codes over Certain Finite Fields. *Journal of the Society for Industrial and Applied Mathematics*, 1960.
- [41] Francisco Romero, Gohar Irfan Chaudhry, Íñigo Goiri, Pragna Gopa, Paul Batum, Neeraja J Yadwadkar, Rodrigo Fonseca, Christos Kozyrakis, and Ricardo Bianchini. FaaS: A transparent auto-scaling cache for serverless applications. In *Proc. of ACM SoCC*, 2021.
- [42] Maheswaran Sathiamoorthy, Megasthenis Asteris, Dimitris Papailiopoulos, Alexandros G Dimakis, Ramkumar Vadali, Scott Chen, and Dhruba Borthakur. XORing Elephants: Novel Erasure Codes for Big Data. In *Proc. of ACM VLDB*, 2013.
- [43] Nihar B Shah, KV Rashmi, P Vijay Kumar, and Kannan Ramchandran. Interference alignment in regenerating codes for distributed storage: Necessity and code constructions. *IEEE Trans. on Information Theory*, 58(4):2134–2158, 2012.
- [44] Yingdi Shan, Kang Chen, Tuoyu Gong, Lidong Zhou, Tai Zhou, and Yongwei Wu. Geometric partitioning: Explore the boundary of optimal erasure code repair. In *Proc. of ACM SOSP*, 2021.
- [45] Itzhak Tamo, Zhiying Wang, and Jehoshua Bruck. Zigzag Codes: MDS Array Codes with Optimal Rebuilding. *IEEE Trans. on Information Theory*, 59(3):1597–1616, 2013.
- [46] Kaicheng Tang, Keyun Cheng, Helen HW Chan, Xiaolu Li, Patrick P. C. Lee, Yuchong Hu, Jie Li, and Ting-Yi Wu. Balancing repair bandwidth and sub-packetization in erasure-coded storage via elastic transformation. In *Proc. of IEEE INFOCOM*, 2023.
- [47] Twitter. Blobstore: Twitter’s in-house photo storage system. <https://blog.x.com/engineering/en-us/a/2012/blobstore-twitter-s-in-house-photo-storage-system>, Dec 2012.
- [48] Myna Vajha, Vinayak Ramkumar, Bhagyashree Puranik, Ganesh Kini, Elita Lobo, Birenjith Sasidharan, P Vijay Kumar, Alexandar Barg, Min Ye, Srinivasan Narayana-murthy, et al. Clay codes: Moulding MDS codes to yield an MSR code. In *Proc. of USENIX FAST*, 2018.
- [49] Hakim Weatherspoon and John D Kubiatowicz. Erasure coding vs. replication: A quantitative comparison. In *International Workshop on Peer-to-Peer Systems*, pages 328–337. Springer, 2002.

A Proof Appendix

To prove NCBlob works for $n - k = 3$ (§3.3), we extend the proof for F-MSR codes with $n - k = 2$ in [19]. To this end, we introduce two definitions: *Repair-based collections* (RBCs) and *evolved repair MDS property* (erMDS), which are first proposed in [19], to prove the correctness of $(k + 3, k)$ NCBlob.

Specifically, a RBC in $(k + 3, k)$ NCBlob refers to a collection of $3k$ sub-blocks from nodes; if a RBC can decode all k data blocks, then it is *decodable*. We define three kinds of RBCs of the r^{th} repair, including $RBC_1^r : (1, 1, 1, 3, \dots, 3)$, $RBC_2^r : (1, 2, 3, \dots, 3)$ and $RBC_3^r : (2, 2, 2, 3, \dots, 3)$, where RBC_1^r refers to the $3k$ sub-blocks collected from any three nodes (each provides one sub-block) and any other $(k - 1)$ nodes (each provides three sub-blocks) after the r^{th} repair, and RBC_2^r and RBC_3^r have similar meanings. Based on results in [19], the repair MDS property after r^{th} repair is equivalent to ensuring that all possible RBC_1^r 's are decodable. In this paper, we let the erMDS property after r^{th} repair refer to ensuring that all possible RBC_2^r 's and RBC_3^r 's are decodable.

Our idea to give NCBlob's correctness is to prove that the MDS, repair MDS and erMDS properties are met after the r^{th} repair, provided that they have been met before the r^{th} repair, formulated as Lemmas 1, 2 and 3, respectively, as follows.

Lemma 1. [19] *NCBlob meets the MDS property after the r^{th} repair, if the repair MDS property is met before the r^{th} repair.*

Lemma 2. *NCBlob meets the repair MDS property after the r^{th} repair, if the erMDS property is met before the r^{th} repair.*

Proof. We prove by induction on r . Initially, $(k + 3, k)$ NCBlob is RS coded (§3.3), so any $3k$ sub-blocks of NCBlob are decodable. Thus, $(k + 3, k)$ NCBlob meets the repair MDS property when $r = 0$ (induction hypothesis), as well as the erMDS property, i.e., all possible RBC_2^0 's and RBC_3^0 's are decodable.

In the r^{th} ($r \geq 1$) repair, take $r = 1$ for instance. Suppose that $(6, 3)$ NCBlob stores $p_{j,0}, p_{j,1}$ and $p_{j,2}$ in N_j ($0 \leq j \leq 5$) and N_0 fails, and repairs the lost three sub-blocks $p'_{0,0}, p'_{0,1}$ and $p'_{0,2}$ in a new node N'_0 by selecting one sub-block from each helper, say $p_{1,0}, p_{2,0}, \dots, p_{5,0}$. Let $\mathcal{U}_1 = \{p'_{0,0}, p'_{0,1}, p'_{0,2}; p_{1,0}, p_{1,1}, p_{1,2}; \dots; p_{5,0}, p_{5,1}, p_{5,2}\}$ be the set of all sub-blocks after the repair. Next we prove that $\mathcal{U}_1$ can meet the repair MDS property; i.e., all possible RBC_1^1 can be decodable. There are two cases to construct any RBC_1^1 from $\mathcal{U}_1$.

Case 1: RBC_1^1 collects one sub-block (say $p'_{0,0}$) from N'_0 . Since RBC_2^0 's are decodable, say, one of which is $\mathcal{D}_1 = \{p_{1,0}; p_{2,0}, p_{2,1}; p_{3,0}, p_{3,1}, p_{3,2}; p_{4,0}, p_{4,1}, p_{4,2}\}$. Then, we can always construct a corresponding RBC_1^1 as $\mathcal{R}_1 = \{p'_{0,0}; p_{1,0}; p_{2,0}; p_{3,0}, p_{3,1}, p_{3,2}; p_{4,0}, p_{4,1}, p_{4,2}\}$, satisfying that $\mathcal{R}_1 = \mathcal{D}_1 - \{p_{2,1}\} + \{p'_{0,0}\}$.

Due to the decodability of $\mathcal{D}_1$, we can let $p'_{0,0} = \gamma_1 \times \mathcal{D}_1$, where γ_1 denotes an encoding coefficient vector. Since $p_{2,1} \in \mathcal{D}_1$, we can have $\text{span}(\mathcal{R}_1) = \text{span}(\mathcal{D}_1 - \{p_{2,1}\} + \{\gamma_1 \times \mathcal{D}_1\}) = \text{span}(\mathcal{D}_1)$, which makes $\mathcal{R}_1$ decodable. Thus, all possible RBC_1^1 's in Case 1 are decodable.

Case 2: RBC_1^1 collects all three sub-blocks from N'_0 . We can also use the decodability in RBC_3^0 to make all possible RBC_1^1 's in Case 2 decodable (similar to Case 1 and thus omitted).

Combining Case 1 and Case 2 makes Lemma 2 hold. $\square$

Lemma 3. *NCBlob meets the erMDS property after the r^{th} repair, if the erMDS property is met before the r^{th} repair.*

Proof. We prove by induction on r via a $(6, 3)$ NCBlob example like Lemma 2. $(6, 3)$ NCBlob meets the erMDS property when $r = 0$ (induction hypothesis), as stated in Lemma 2. To show how the rotation based repair (§3.3) works, we consider two consecutive repairs. Take $r = 1$ and $r = 2$ for instance.

The first repair ($r = 1$). We consider the same case in Lemma 2 for $r = 1$ and obtain the same $\mathcal{U}_1$. Next we need to prove that $\mathcal{U}_1$ still can meet the erMDS property; i.e., all possible RBC_2^1 's and RBC_3^1 's can be decodable. First, we consider RBC_2^1 and there are three cases to construct a RBC_2^1 from $\mathcal{U}_1$.

Case 1: RBC_2^1 selects one sub-block (say $p'_{0,0}$) from N'_0 . Since all possible RBC_2^0 's are decodable, say, one of which is $\mathcal{D}_2 = \{p_{1,0}, p_{1,1}; p_{2,0}, p_{2,1}, p_{2,2}; p_{3,0}, p_{3,1}, p_{3,2}; p_{4,0}\}$. Then, we can always construct a corresponding RBC_2^1 as $\mathcal{R}_2 = \{p'_{0,0}; p_{1,0}, p_{1,1}; p_{2,0}, p_{2,1}, p_{2,2}; p_{3,0}, p_{3,1}, p_{3,2}\}$, satisfying that $\mathcal{R}_2 = \mathcal{D}_2 - \{p_{4,0}\} + \{p'_{0,0}\}$. Due to the decodability of $\mathcal{D}_2$, we can also ensure that $\mathcal{R}_2$ is decodable, similarly to the proof in Lemma 2. Thus, RBC_2^1 's in Case 1 are decodable.

Case 2 and Case 3: Similar to Case 1 and omitted.

Combining all Cases, all possible RBC_2^1 's are decodable. Similarly to the proof for RBC_2^1 , all possible RBC_3^1 's are also decodable. Thus, the erMDS property holds for $r = 1$.

The second repair ($r = 2$). Suppose that N_1 fails. Then we select the second row of sub-blocks in each helper which are rotated from those in the first repair, i.e., $\{p'_{0,1}, p_{2,1}, \dots, p_{5,1}\}$ to repair $p''_{1,0}, p''_{1,1}, p''_{1,2}$ in a new node N''_1 . Let $\mathcal{U}_2 = \{p'_{0,0}, p'_{0,1}, p'_{0,2}; p''_{1,0}, p''_{1,1}, p''_{1,2}; \dots; p_{5,0}, p_{5,1}, p_{5,2}\}$ be the set of all sub-blocks after the second repair. Next we prove that $\mathcal{U}_2$ can meet the erMDS property; i.e., all possible RBC_2^2 's and RBC_3^2 's can be decodable. First, we consider RBC_2^2 and there are three cases to construct a RBC_2^2 from $\mathcal{U}_2$.

Case 1: RBC_2^2 selects one sub-block from N''_1 (say $p''_{1,1}$). Since all possible RBC_2^1 's are decodable, say, one of which is $\mathcal{D}_3 = \{p_{2,1}, p_{2,2}; p_{3,0}, p_{3,1}, p_{3,2}; p_{4,0}, p_{4,1}, p_{4,2}; p_{5,1}\}$. Note that $\mathcal{D}_3$ is rotated from $\mathcal{D}_2$ by a row which enables us to always construct $\mathcal{D}_{r+1}$ based on $\mathcal{D}_r$ for each repair iteration. Then, we can always construct a corresponding RBC_2^2 as $\mathcal{R}_3 = \{p''_{1,1}; p_{2,1}, p_{2,2}; p_{3,0}, p_{3,1}, p_{3,2}; p_{4,0}, p_{4,1}, p_{4,2}\}$, satisfying that $\mathcal{R}_3 = \mathcal{D}_3 - \{p_{5,1}\} + \{p''_{1,1}\}$. Due to the decodability of $\mathcal{D}_3$, we can ensure that $\mathcal{R}_3$ is decodable, similarly to Lemma 2. Thus, RBC_2^2 's in Case 1 are decodable.

Case 2 and Case 3: Similar to Case 1 and omitted.

Combining all Cases, all possible RBC_2^2 are decodable. Similarly to the proof for RBC_2^2 , all possible RBC_3^2 's are also decodable. Thus, the erMDS property holds for $r = 2$. $\square$

Theorem 1. $(k + 3, k)$ NCBlob always exists.

Proof. Based on Lemmas 1, 2 and 3, Theorem 1 holds. $\square$

B Artifact Appendix

Abstract

Our artifact includes the prototype implementation of NCBlob and other state-of-the-art comparison erasure coding schemes. We explain how to prepare, install, configure NCBlob to re-run the experiments in this paper.

Scope

Our artifact has been made *available* and has two main goals: The first is to enable the validation of the main claims presented in the paper. The second is to facilitate others in building experiments for their own erasure coding projects.

Contents

The artifact is hosted in a git repository. This repository includes NCBlob's source code as well as documentation and example applications. It is structured as follows:

Source code

./common: This folder contains generic utility data structures for multi-threaded communication and binary data storage.

./core: This folder contains the source code of the system's core logic for the coordinator and workers (i.e., `DataServers` in §4), mainly responsible for the generation and transmission of data flows and control flows.

./ec: This folder contains the code of coding module which includes specific encoding algorithms (RS, Clay, F-MSR and NCBlob).

./lib: This folder contains dependencies on third-party external libraries, such as the encoding library `jerasure`, the binary object storage library `blob_store`, and the thread pool `BS::thread_pool`.

./meta: This folder contains the source code for generating and storage of metadata for objects and stripes.

./protocol: This folder contains the source code of inter-node communication protocols, which define the data structures of control flows and data flows.

./script: This folder contains scripts to automatically prepare and launch the coordinator and workers of NCBlob.

./task: This folder contains the source code of specific generation logic for control flows.

./trace: This folder contains the code to generate the evaluation workloads based on the trace, along with the merging schemes.

./coordinator.cc: Entry point of the coordinator program.

./data_worker.cc: Entry point of the worker program.

Configuration

./coord_cfg.toml: Configuration file for the coordinator.

./worker_cfg.toml: Configuration file for the worker.

Hosting

NCBlob artifact repository is hosted on GitHub and archived using Zenodo with a permanent DOI.

- **Repository:** <https://github.com/YuchongHu/NCBlob>.
- **Zenodo Archive:** <https://zenodo.org/records/14684822>.
- **DOI:** <https://zenodo.org/doi/10.5281/zenodo.14684821>.

Requirements

Hardware dependencies

Twenty-two machines are recommended to run the experiments with our prototype and evaluation scripts. These machines need to be connected via a network, so that they are reachable from each other. For each machine, we recommend an eight-core CPU, 16GB of memory and above, and an HDD.

We need at least six machines to form the storage cluster with the default erasure coding parameters $(n, k) = (6, 4)$. In addition, we need one machine to run the coordinator and one machine to run the client.

Software dependencies

Our artifact is developed and tested on Ubuntu 22.04 LTS. The following software packages are required:

- **Basic:** `gcc`, `g++`, `boost`.
- **Serialization and storage:** `msgpack-cxx`, `leveldb`, `hiredis`.
- **Logging and formatting:** `glog`, `fmt`.

Testbed Setup

Users can prepare the testbed with the following steps:

- Prepare the hardware and software dependencies as described above.
- Download and extract the source code of our artifact from URL: <https://github.com/YuchongHu/NCBlob>.
- Build the artifact project with the following steps: (1) make a build directory `mkdir ./build && make ./bin && cd ./build`; (2) make the project `cmake -DCMAKE_BUILD_TYPE=RELEASE .. && cmake --build . --target all`.
- Configure and run the coordinator and worker nodes with the configuration files `coord_cfg.toml` and `worker_cfg.toml`.

For the detailed setup, configuration and other instructions, please refer to the README file in the artifact repository.